```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *head1 = NULL;
struct Node *head2 = NULL;
struct Node* createList(int n) {
    struct Node *head = NULL, *newNode, *temp;
    int data;
    if (n <= 0) {
        printf("Number of nodes should be greater than 0\n");
        return NULL;
    }
    for (int i = 1; i <= n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        if (newNode == NULL) {
            printf("Memory allocation failed\n");
            return head;
        }
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        newNode->data = data;
        newNode->next = NULL;
        if (head == NULL)
            head = newNode;
        else
            temp->next = newNode;

        temp = newNode;
    }
    printf("Linked list created successfully\n");
    return head;
}

void displayList(struct Node *head) {
    struct Node *temp = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Linked List: ");
    while (temp != NULL) {
```

```
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void sortLinkedList(struct Node *head) {
    struct Node *i, *j;
    int tempData;
    if (head == NULL) {
        printf("List is empty, cannot sort.\n");
        return;
    }
    for (i = head; i->next != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                tempData = i->data;
                i->data = j->data;
                j->data = tempData;
            }
        }
    }
    printf("Linked list sorted successfully\n");
}
struct Node* reverseLinkedList(struct Node *head) {
    struct Node *prev = NULL, *curr = head, *next = NULL;
        while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    printf("Linked list reversed successfully\n");
    return prev;
}

struct Node* concatenateLinkedList(struct Node *head1, struct Node *head2) {
    struct Node *temp;
    if (head1 == NULL)
        return head2;
    temp = head1;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = head2;
```

```
    printf("Linked lists concatenated successfully\n");
    return head1;
}
int main() {
    int choice, n;
    while (1) {
        printf("1. Create List 1\n");
        printf("2. Create List 2\n");
        printf("3. sort List 1\n");
        printf("4. reverse List 1\n");
        printf("5. Sort List 2\n");
        printf("6. Reverse List 2\n");
        printf("7. Concatenate\n");
        printf("8. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            printf("Enter number of nodes for List 1: ");
            scanf("%d", &n);
            head1 = createList(n);
            break;
        case 2:
            printf("Enter number of nodes for List 2: ");
            scanf("%d", &n);
            head2 = createList(n);
            break;
        case 3:
            sortLinkedList(head1);
            displayList(head1);
            break;
        case 4:
            head1 = reverseLinkedList(head1);
            displayList(head1);
            break;
        case 5:
            sortLinkedList(head2);
            displayList(head2);
            break;
        case 6:
            head2= reverseLinkedList(head2);
            displayList(head2);
            break;
```

```
        case 7:
            head1 = concatenateLinkedList(head1, head2);
            printf("After concatenation:\n");
            displayList(head1);
            break;
        case 8:
            printf("Exiting program...\n");
            exit(0);
        default:
            printf("Invalid choice. Try again.\n");
        }
    }
    return 0;
}
```

```
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 1
Enter number of nodes for List 1: 2
Enter data for node 1: 10
Enter data for node 2: 5
Linked list created successfully
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 2
Enter number of nodes for List 2: 2
Enter data for node 1: 50
Enter data for node 2: 89
Linked list created successfully
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 3
Linked list sorted successfully
Linked List: 5 -> 10 -> NULL
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 4
Linked list reversed successfully
```

```
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 5
Linked list sorted successfully
Linked List: 50 -> 89 -> NULL
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 6
Linked list reversed successfully
Linked List: 89 -> 50 -> NULL
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 7
Linked lists concatenated successfully
After concatenation:
Linked List: 10 -> 5 -> 89 -> 50 -> NULL
1. Create List 1
2. Create List 2
3. sort List 1
4. reverse List 1
5. Sort List 2
6. Reverse List 2
7. Concatenate
8. Exit
Enter choice: 8
Exiting program...

Process returned 0 (0x0)   execution time : 796.467 s
```